

CSE 331: Computer Security

Fundamentals

Spring 2026

Symbolic Execution

Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

安徽大學紐約石溪學院

Bugs are difficult to catch

- Rare features
- Rare circumstances
- Nondeterminism

Last class: Static Analysis

- Analyze all possible runs of a program
- Can it find deep, difficult bugs?
 - Theoretically, yes. Practically, not so much.
- Precision vs. Scalability
- flow/context/path-sensitivity add precision (still imperfect) but reduce speed/scalability.
- Difficult for developers to reason about the errors: too many warnings, no test cases

Core issue: Abstraction

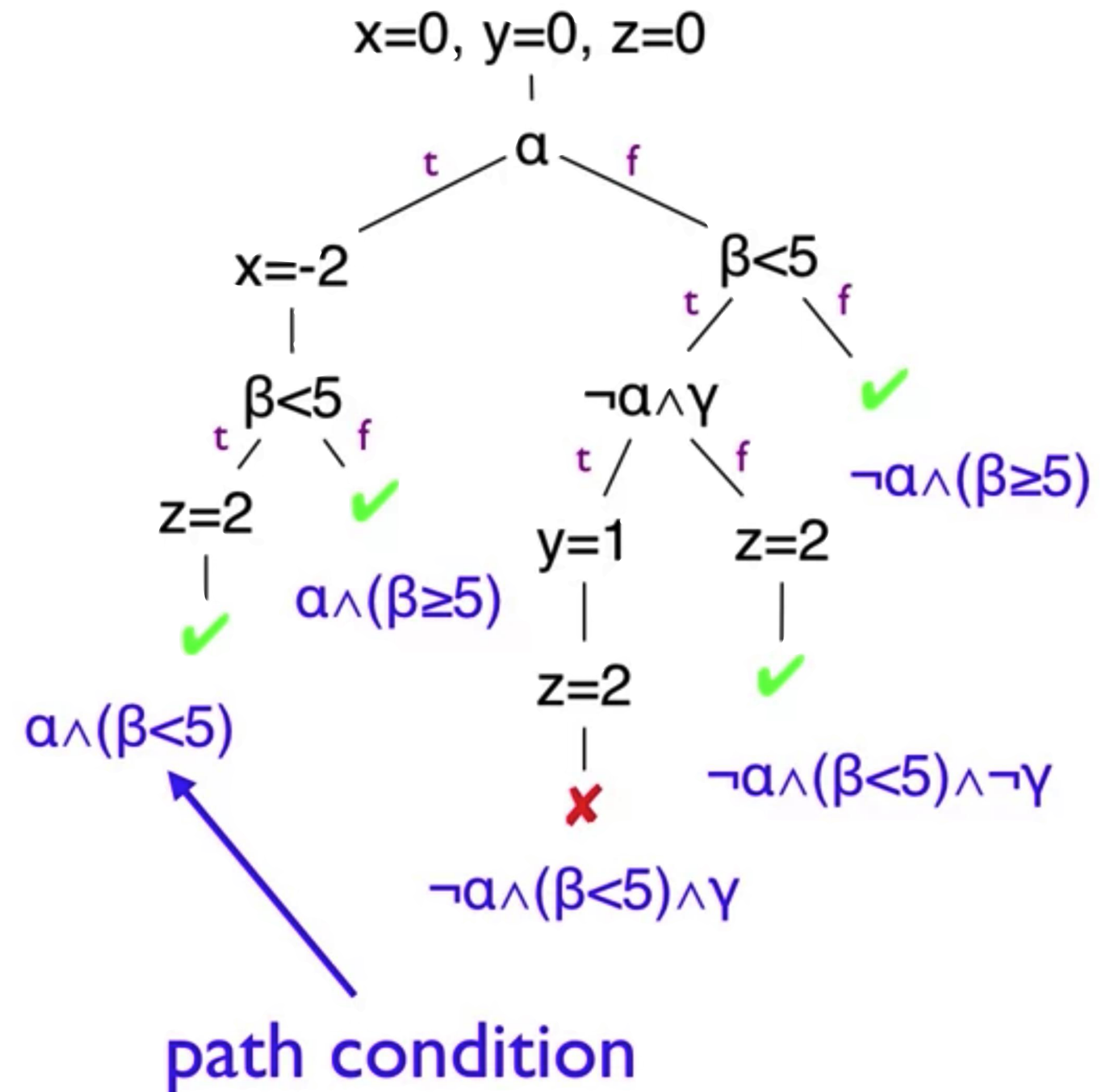
- Abstraction lets us model all possible runs: Boolean instead of numerical value evaluation
 - But abstraction introduces conservatism
- *-sensitivities add precision, to deal with this
 - * = flow-, context-, path-, information flow-, etc.
 - But more precise abstractions are more expensive
 - Challenges scalability
 - Still have false alarms or missed bugs
- Static analysis abstraction != developer abstraction
 - Because the developer didn't have them in mind

Symbolic Execution

- Middle ground between testing and static analysis
- Testing is great (do it!)
`assert(f(3) == 5)`
 - but each test only explores one possible execution.
 - Complete, unsound.
 - Don't necessarily generalize.
- Symbolic execution generalizes testing.
`y=α; assert(f(y) == 2 * y - 1)`
 - More sound
 - If execution path depends on unknown variables, conceptually forks the run

Simple Example

```
1. int a = α, b = β, c = γ;  
2. // symbolic  
3. int x = 0, y = 0, z = 0;  
4. if (a) {  
5.   x = -2;  
6. }  
7. if (b < 5) {  
8.   if (!a && c) { y = 1; }  
9.   z = 2;  
10. }  
11. assert(x+y+z != 3)
```



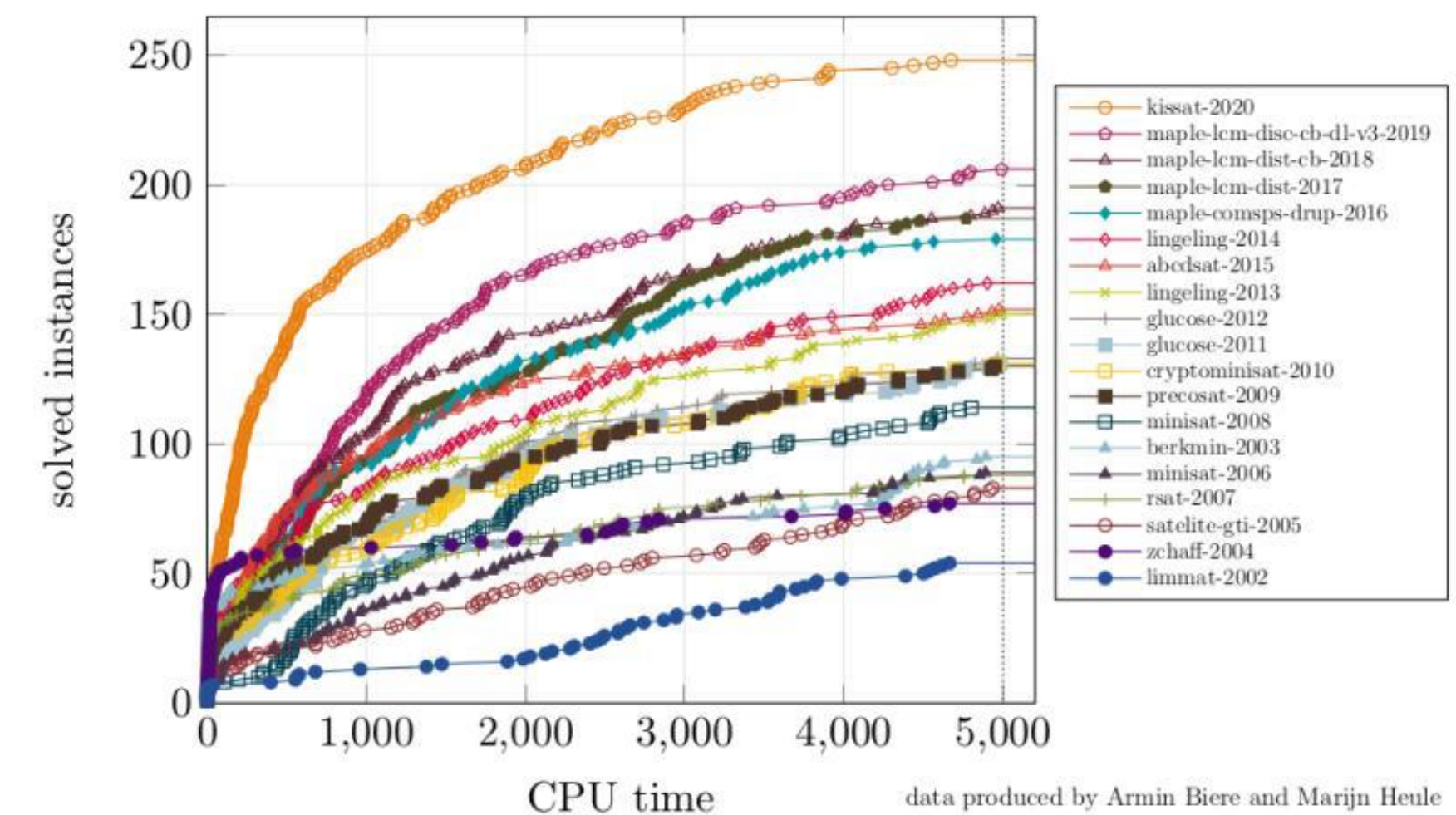
Insight

- Each symbolic execution path stands for many actual program runs.
- Much better code coverage.
- Complete but not sound
- Programs don't usually have a finite number of paths and even if they do, there are just too many.
- All our static analysis tools help us to improve precision

Why wasn't it adopted ages ago?

- Symbolic Execution has been around since 1976
- Compute-intensive
 - Many paths
 - Tons of queries to solver to decide feasibility of each path and test assertions
 - Large program state space
- Computers were slow
- Moore's law: Exponential speed up
- Better algorithms: SMT/SAT solvers are still active areas of research

SAT Competition Winners on the SC2020 Benchmark Suite



Symbolic Execution

- Grammar of expressions:
 - Integers
 - Variables
 - Arithmetic and logical operations
 - Symbolic variables $\rightarrow$ represent inputs
 - Solve for symbolic expressions

Symbolic Expressions

- Make a language interpreter to be able to **compute symbolically**
 - Normally, a program's variables contain values
 - Now they can also contain symbolic expressions
 - Normal values: 5, “hello”, $a[2]$, ...
 - Symbolic expressions $\alpha+5$, “hello”+ α , $a[\alpha+\beta+2]$, ...

Simple Example

```
x = read();  
y = 5 + x;  
z = 7 + y;  
a[z] = 1;
```

Concrete Memory

$x \mapsto 5$

$y \mapsto 10$

$z \mapsto 17$

$a \mapsto \{0, 0, 0, 0\}$

Overrun!

Symbolic Memory

$x \mapsto \alpha$

$y \mapsto 5 + \alpha$

$z \mapsto 12 + \alpha$

$a \mapsto \{0, 0, 0, 0\}$

Possible overrun!

Path Condition

```
1  x = read();  
2  if (x>5) {  
3      y = 6;  
4      if (x<10)  
5          y = 5;  
6  } else y = 0;
```

- We can represent the influence of symbolic values on the current path by using a path condition π :

- Line 3 — $\pi: \alpha > 5$
- Line 5 — $\pi: \alpha > 5$ and $\pi: \alpha < 10$
- Line 6 — $\pi: \alpha \leq 5$

Path not feasible?
path condition won't be satisfiable

- Solutions to path constraints can be used as test cases for the path

Paths and Assertions

```
1 x = read();  
2 y = 5 + x;  
3 z = 7 + y;  
4 if(z < 0)  
5     abort()  
6 if(z >= 4);  
7     abort();  
8 a[z] = 1;
```

$\pi = \text{true}$

$\pi = \text{true}$

$\pi = \text{true}$

$\pi = \text{true}$

$\pi = 12 + \alpha < 0$

$\pi = \neg(12 + \alpha < 0)$

$\pi = \neg(12 + \alpha < 0) \wedge 12 + \alpha \geq 4$

$\pi = \neg(12 + \alpha < 0) \wedge \neg(12 + \alpha \geq 4)$

If either line 5 or 7 is reached,
we have an out of bound access

Forking Execution

- Symbolic executor can fork at branching points.
- There is a solution to both the path condition and its negation.
- How can we systematically explore both?
 - Check feasibility during execution —> Calling solver is expensive
 - Randomly choose path —> Extra unnecessary work?
 - **Concolic (Concrete + Symbolic) execution:** Run the program concretely while maintaining symbolic state. Concrete execution chooses the path to explore first.

Symbolic Execution Algorithm

- Create initial task: $pc = 0, \pi = \emptyset, \sigma = \emptyset$
- Add task to worklist: (pc, π, σ)
- while tasklist is not empty:
 - pull some task from worklist
 - execute. if fork at (pc_0, π_0, σ_0) :
 - add task $(pc_1, (\pi_0 \wedge p), \sigma_0)$ (assuming $\pi_0 \wedge p$ is feasible)
 - add task $(pc_2, (\pi_0 \wedge \sim p), \sigma_0)$ (assuming $\pi_0 \wedge \sim p$ is feasible)

```
pc0    if (p) {  
pc1      ...  
pc2 } else { ...
```

Practical Considerations

- How to deal with libraries/native code?
 - Pull that code in!
 - Often infeasible
 - Analyze simpler versions of the library as an alternative (libc->newlib)
 - Create model of the code

Concolic Execution

(AKA Dynamic Symbolic Execution)

- **Instrument the program** to do symbolic execution as it runs
 - shadow concrete program state with symbolic variables
 - initial concrete state determines initial path
 - could be randomly generated
 - **keep shadow path condition**
- **Explore one path at a time, start to finish**
 - Choose next path base on:
 - Negating some elements of the last path condition
 - Solving for it to produce concrete inputs for the next test
 - Always have a concrete underlying value to rely on

Concretization

- Replace symbolic variables with concrete values that satisfy the path condition.
 - Maintain them as the program runs
- Allows you to do system calls, library calls, etc.
 - Just lose symbolic-ness!
- Allow you to reduce complexity at will.

Search, SAT, and SMT

- Symbolic execution is simple but expensive
- Boils down to a search problem
- Rise of SMT (Satisfiability Modulo Theories) solvers

SAT (Satisfiability) Solver: can you find **satisfying assignment** for a logical expression like **$(a \vee !b) \wedge (!a \vee c)$** ?

SMT (Satisfiability Modulo Theories) Solver: can you find a solution for a system of equations like **$(3x - 2y = 1) \vee (y - x = 1) \wedge (x * y = 25)$** ?

Path Explosion

```
1. int a =  $\alpha$ , b =  $\beta$ , c =  $\gamma$ ; // symbolic  
2. if (a) ... else ...;  
3. if (b) ... else ...;  
4. if (c) ... else ...;
```

```
1. int a =  $\alpha$ ; // symbolic  
2. while (a) do ...;  
3. ...
```

- Too many paths to run symbolic execution exhaustively
- Exponential in branching
- Loops are even worse

3 variables, 8 paths

Potentially 2^{31} paths

Search strategies

- Think of a program as a Directed acyclic graph (DAG)
 - Nodes = program states, Edges = transitions
 - Graph exploration algorithm
- Simple ideas:
 - Depth-first search (DFS): Worklist = stack
 - Breadth-first search (BFS): Worklist = queue
- Drawbacks: not guided by any knowledge
 - DFS can get stuck easily.
 - BFS fails to go deep (harder to make concolic)

Search strategies

- Need prioritization to steer search toward more interesting paths
- Need time limit/condition of termination
- Some ideas:
 - **Random path:** pick next path to explore uniformly at random
 - **Random restart:** explore for some time then restart if you didn't find anything interesting
 - **Define a prioritization strategy:** define some sort of heuristic pick either the maximum or based on weighted probability.
- **Reproducibility** becomes a challenge
 - Use pseudo-randomness

Heuristic: Coverage

- Try to achieve maximum coverage: Visit statements you haven't seen before.
 - Keep a counter for each statements (# of visits), pick the next lowest current score.
 - Pro: tries to reach everywhere. Hard to reach places are more prone to error
 - Con: based on pre-conditions, you may not be able to get there

Hueristic: Generational Search

- Hybrid of BFS (Breadth-first search) and coverage-guided
- Generation 0: Pick one program at random, run it to completion
- Generation $n+1$: Take paths from gen n ; negate one branch condition on a path to get a new path prefix; find a solution for that prefix; take the path.
 - For any variable not constrained by the prefix, do semi-random assignment.
- Use coverage heuristic to prioritize among paths.
 - Works well with concolic execution

Combined Search

- Run multiple searches at the same time, alternate between them.
- Large overhead
- Potentially better reach.

How SAT solver works

- Database of constraints in CNF form (ANDs of ORs)
- Pick a constraints; set it.
- Apply it to all available clauses.
- Continue to either finish assignment successfully or reach a contradiction

SMT Solver

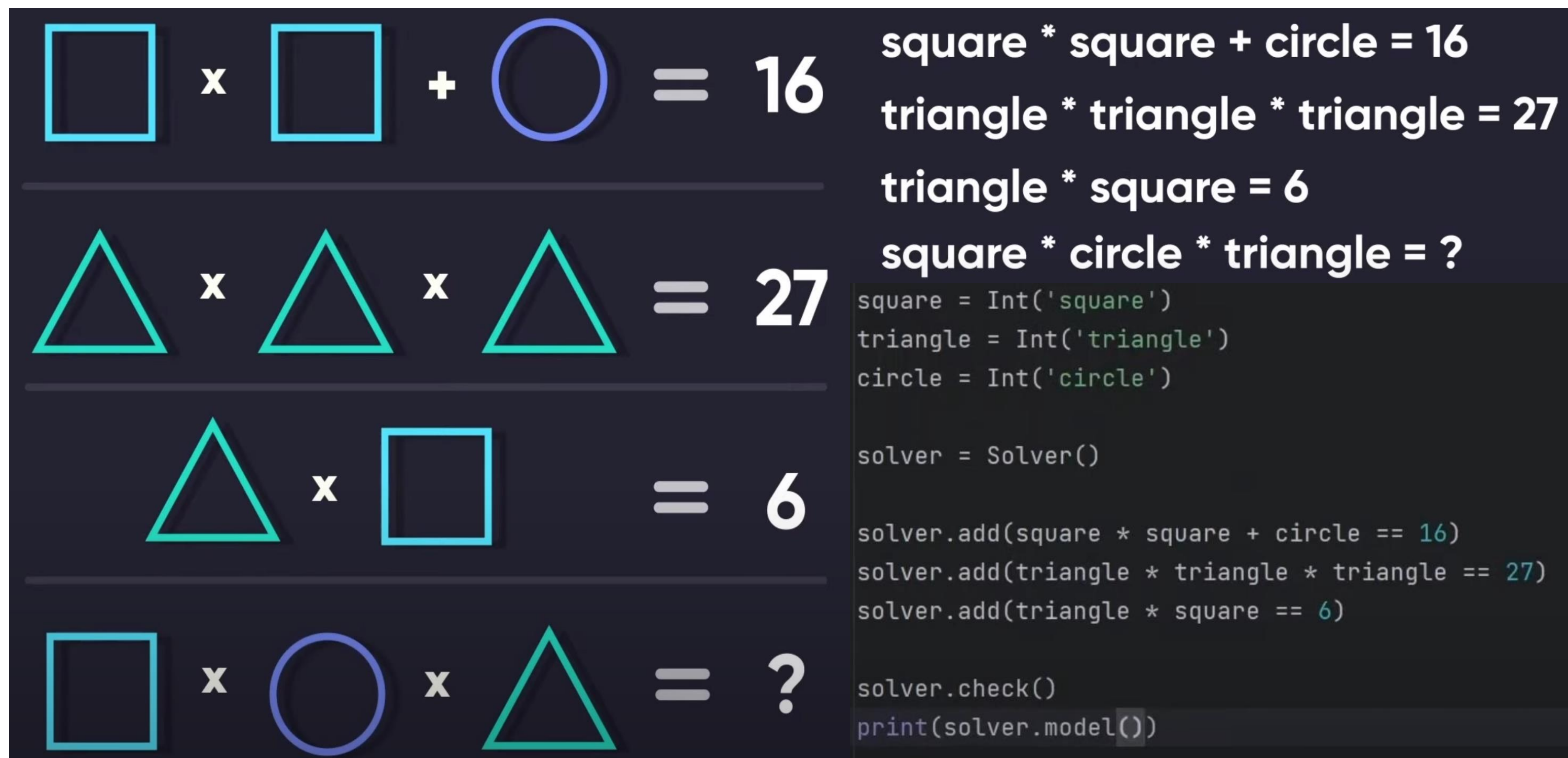
- Takes Logic formula
- Tells you whether it is satisfiable or not
- if it is, gives you answer
- SMT solvers are SAT solvers at their core
 - In theory, we can reduce all SMT queries to SAT queries
 - In Practice, SMT has many optimizations

SMT Solver Performance

- Tons of optimizations in engines and translators
 - Caching
 - Removing redundant variables
 - Simulate memory
 - Deal with arrays, arithmetics solver

Microsoft Z3

- Widely used in software and hardware verification, program analysis, constraint solving, and symbolic execution.



The image displays a visual representation of a constraint solving problem using shapes and a corresponding Z3 Python script. The visual part consists of four rows of equations, each separated by a horizontal line. The first row shows two squares multiplied together plus a circle equals 16. The second row shows three triangles multiplied together equals 27. The third row shows a triangle multiplied by a square equals 6. The fourth row shows a square multiplied by a circle multiplied by a triangle equals a question mark. The shapes are colored: squares are light blue, triangles are light green, and the circle is light purple. The Z3 script on the right defines variables for square, triangle, and circle, adds the constraints from the equations, and checks for a solution.

Visual equations:

- $\square \times \square + \bigcirc = 16$
- $\triangle \times \triangle \times \triangle = 27$
- $\triangle \times \square = 6$
- $\square \times \bigcirc \times \triangle = ?$

Z3 Python script:

```
square = Int('square')
triangle = Int('triangle')
circle = Int('circle')

solver = Solver()

solver.add(square * square + circle == 16)
solver.add(triangle * triangle * triangle == 27)
solver.add(triangle * square == 6)

solver.check()
print(solver.model())
```

Path based search is ultimately limited

```
int counter = 0, values = 0;
for (i = 0; i < 100; i++) {
    if (input[i] == 'B') {
        counter++;
        values += 2;
    }
}
assert(counter != 75);
```

2^{100} possible execution paths

SAGE

(Scalable, Automated, Guided Execution)

- **Concolic executor** developed at **Microsoft Research**

- Grew out of Godefroid's work on DART
- Uses generational search

Never publicly released;

- Primarily **targets bugs in file parsers**

- E.g., JPEG, DOCX, PPT, etc
- Good fit for concolic execution
 - Likely to terminate
 - Just input/output behavior

Leads to open-source implementations like RiverConc

Now: LLM + Symbolic Execution Tools to handle path explosion

Impact: since 2007

- 500+ machine years (in largest fuzzing lab in the world)
- 3.4 Billion+ constraints (largest SMT solver usage ever!)
- 100s of apps, 100s of bugs (missed by everything else...)
- Ex: **1/3** of **all** Win7 WEX security bugs found by SAGE →
- Bug fixes shipped quietly (no MSRCs) to 1 Billion+ PCs
- Millions of dollars saved (for Microsoft and the world)
- SAGE is now used daily in Windows, Office, etc.

